CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 13: Review Configurations

> **Exam objective:** Design Claude Code review configurations that load the correct project standards, restrict unnecessary tool access, and produce structured output suitable for automated downstream processing.

Q1.

A repository has strict internal review rules documented in `docs/review-standards.md`. A team wants every Claude Code review run to apply those rules consistently. What is the best configuration approach?

A. Assume the model will infer the standards from the codebase without explicit loading B. Configure the review to load the project-specific standards document as part of its instructions/context C. Replace the standards with a generic "follow best practices" note D. Grant more tool access so the model can search the internet for conventions

Answer: B **Explanation:** Review configurations should explicitly load the relevant project standards so the model evaluates changes against the correct expectations. Generic guidance is too vague, and external search is unnecessary when the canonical standards are in-repo. Explicit context improves consistency and traceability.

Q2.

An automated pipeline parses Claude Code review output to open tickets for high-severity issues. Which output style is best?

A. Free-form prose with conversational commentary B. Structured output with stable fields such as severity, file, finding, and recommendation C. A poetic summary followed by optional notes D. Screenshots embedded in markdown

Answer: B **Explanation:** Automated downstream processing depends on predictable structure and stable keys. Structured findings are easier to parse, validate, and route into ticketing or dashboards. Free-form prose is harder to consume reliably at scale.

Q3.

A review task only needs to inspect the current diff and repository guidelines. Which tool-access policy is most appropriate?

A. Allow all tools, including network and unrestricted shell, in case the model becomes curious B. Restrict access to only the tools needed to read the diff and referenced standards C. Deny access

to the repository so bias is reduced D. Enable write access so the model can silently fix defects instead of reporting them

Answer: B **Explanation:** Least privilege is the right design for review workflows. If the goal is read-only inspection of diffs and standards, only those capabilities should be enabled. Broader access increases security and operational risk without improving the review objective.

Q4.

A company has separate standards for backend and frontend repositories. How should Claude Code review configurations handle this?

A. Use a single universal review prompt for all repositories B. Load the standard that matches the repository or project type being reviewed C. Omit standards to avoid overfitting D. Ask the model to choose whichever standard seems best during runtime

Answer: B **Explanation:** Review quality improves when the configuration loads the standards that actually apply to the project under review. Different stacks often have different conventions and risk models. Universal prompts or runtime guessing reduce precision.

Q5.

A compliance team wants review results to feed a policy engine that rejects builds when certain issue classes are present. Which requirement is most important?

A. The output must include machine-readable categories or codes that the policy engine can evaluate deterministically B. The review must be written in natural language so humans enjoy reading it C. The configuration must maximize tool access D. The model must be allowed to rewrite code automatically

Answer: A **Explanation:** Policy engines need deterministic, machine-readable signals such as categories, severities, or rule IDs. Human-friendly prose can still exist, but it is secondary when automated gating is required. Broader access and auto-remediation are unrelated to the stated need.

Q6.

Why should a review configuration explicitly avoid unnecessary write-capable tools when the task is only to assess changes?

A. Because write access can expand the blast radius without improving a read-only review task B. Because models cannot analyze code unless they can edit it C. Because structured output is incompatible with read-only tools D. Because disabling writes automatically makes findings more severe

Answer: A **Explanation:** Read-only reviews do not benefit from write access, but write access increases risk through accidental or unauthorized modifications. Principle of least privilege applies directly here. Analytical quality does not depend on having edit capabilities.

Q7.

A team wants review output that can be both shown in CI logs and parsed by a later workflow. What is the best design?

A. Purely narrative markdown paragraphs B. A structured schema, optionally with concise human-readable summaries derived from the same fields C. Binary output to reduce tokens D. Randomized wording to avoid repetition

Answer: B **Explanation:** A schema-first design supports automation, while summaries generated from those structured fields keep the output readable for humans. Narrative-only output is brittle for parsers. Randomness actively harms stability.

Q8.

An organization stores secure coding rules in a dedicated file and wants Claude Code to flag deviations during reviews. Which configuration mistake would most likely reduce accuracy?

A. Loading the secure coding rules into the review context B. Restricting tools to read-only access needed for inspection C. Relying on generic review instructions while omitting the project-specific rule file D. Returning structured findings with severity levels

Answer: C **Explanation:** If the relevant secure coding rules are omitted, the review may fall back to generic heuristics and miss organization-specific requirements. Loading the rules and keeping review access read-only are both sound practices. Structured findings help automation, not hurt accuracy.

Q9.

A review configuration is used in an enterprise monorepo with many language-specific standards. Which design is best?

A. Load all standards for all languages every time, regardless of the files changed B. Load only the standards relevant to the project area or file types being reviewed C. Avoid standards entirely because they may conflict D. Give the model network access so it can search for standards as needed

Answer: B **Explanation:** Targeted context improves relevance and reduces noise. Loading only applicable standards helps the model focus on the correct rules for the changed area without overwhelming it with irrelevant policies. External search is unnecessary when internal standards are already defined.

Q10.

A downstream script fails because Claude Code sometimes labels severity as "critical" and other times as "sev1". What configuration change is most appropriate?

A. Remove severity from the output B. Define a fixed output schema with enumerated allowed severity values C. Ask the script to guess equivalence D. Add more tool permissions

Answer: B **Explanation:** Stable enumerations are essential for reliable automation. A fixed schema prevents output drift and makes validation straightforward. Removing severity discards useful data, and broader permissions do not solve formatting inconsistency.

Q11.

The security team wants reviewers to consider an internal checklist before evaluating infrastructure changes. Which configuration most directly supports that?

A. Include the checklist in the loaded review instructions or referenced standards B. Depend on the model's general knowledge of cloud security C. Increase the token limit without changing the review context D. Enable shell access to inspect the runner OS

Answer: A **Explanation:** If a specific internal checklist matters, it should be explicitly included in the review context so the evaluation can align to it. General knowledge may miss organization-specific expectations. Token increases or OS inspection are not substitutes for the checklist itself.

Q12.

A platform engineer wants review output to trigger different workflows for style issues versus security findings. Which output property is most important?

A. A deterministic issue type or category field for each finding B. Longer explanations for every comment C. A separate output file per programming language D. Permission to auto-fix low-severity issues

Answer: A **Explanation:** Workflow routing depends on being able to classify findings consistently. A deterministic type/category field enables separate handling for style, correctness, or security issues. Explanation length and auto-fix permissions are secondary to reliable classification.

Q13.

Why is it valuable to separate review instructions from tool permissions in configuration design?

A. Because the standards define what to evaluate, while permissions define what the agent is allowed to access or do B. Because permissions automatically generate better standards C. Because structured output only works when permissions are unlimited D. Because combining them prevents the model from seeing the diff

Answer: A **Explanation:** Review design has two distinct concerns: evaluation criteria and operational capability. Standards tell the model how to judge changes; permissions constrain the environment and reduce risk. Treating them separately makes configurations safer and more maintainable.

Q14.

A CI job consumes Claude Code review results and posts comments only for findings above a threshold. Which output format best supports that?

A. JSON or similarly structured output with stable severity fields and finding identifiers B. Unstructured markdown with emojis marking importance C. Plain text paragraphs with no labels D.

Screenshots of terminal output

Answer: A **Explanation:** Threshold-based filtering is straightforward when severity and identifiers are structured and stable. Emojis and plain prose are fragile for automation. Structured output is the best fit for programmatic post-processing.

Q15.

A review configuration currently loads the right standards but still allows network and full shell access for every run. What is the best improvement?

A. Remove the standards because they are already loaded B. Narrow tool access to only what the review task actually needs C. Add write permissions for convenience D. Convert the review to an interactive workflow

Answer: B **Explanation:** Even with correct standards, excessive tool access violates least-privilege design and increases risk. Review tasks typically need a small, read-oriented tool set. Tightening access improves safety without sacrificing evaluation quality.